# Propuesta Técnica de EnergyMonitor

Juan Esteban Cortés Parra y Miguel Ángel García Artunduaga

Servicio Nacional de Aprendizaje (SENA)

Tecnólogo en Análisis y Desarrollo de Software (ADSO), ficha 3145556

12 de octubre de 2026 (actualizado el 28 de octubre de 2026)

## Control de versiones

| Versión | Fecha | Descripción del cambio | Responsables |
|---|---|---|---|
| 1.0 | 12/10/2026 | Primera versión: pila tecnológica, arquitectura, infraestructura, costos, cronograma, riesgos y discrepancias. | Cortés Parra, J. E.; García Artunduaga, M. A. |
| 1.1 | 20/10/2026 | Alineación con el sistema desplegado: infraestructura real en producción (VPS Hostinger KVM 2 por un mes, dominio gratuito, Nginx, Let's Encrypt), integración y despliegue continuos con GitHub Actions, costos recalculados, administración global y nuevos eventos, estado por componente, riesgos y discrepancias actualizados. | Cortés Parra, J. E.; García Artunduaga, M. A. |
| 1.2 | 24/10/2026 | Arquitectura en producción: API gateway y microservicios de seguridad, dispositivos y mediciones; el servicio central queda como monolito modular con home, alert, recommendation y notification, cuya migración está en curso; comunicación por REST interno y eventos en RabbitMQ con *transactional outbox*; pila, infraestructura, estado por componente y riesgos actualizados. | Cortés Parra, J. E.; García Artunduaga, M. A. |
| 1.3 | 28/10/2026 | Migración completa: hogares, notificaciones, alertas y recomendaciones como servicios independientes y retiro del servicio central (`energy-monitor-backend`); rutas del gateway, eventos entre servicios (alertas, recomendaciones, actividad de hogares y código de borrado de cuenta), reglas de seguridad de la cuenta (verificación previa del correo, términos y cuentas de Google), integración continua de los ocho repositorios desplegables, pruebas y estado por componente al 28 de octubre. | Cortés Parra, J. E.; García Artunduaga, M. A. |
| 1.4 | 28/10/2026 | Ayuda de la web en Ajustes (documentación por rol, soporte y novedades que el build arma con el historial de git), visor de fotos y fotos de los miembros de un hogar (campo `profileImage` en el perfil interno), alerta de consumo inusual y conteos de pruebas actualizados. | Cortés Parra, J. E.; García Artunduaga, M. A. |

## Resumen

Este documento presenta la propuesta técnica de EnergyMonitor, un sistema que permite a los hogares conocer el consumo eléctrico de cada electrodoméstico, identificar los periodos de mayor demanda, recibir alertas y obtener recomendaciones de ahorro. Describe la pila tecnológica elegida y su justificación, la arquitectura de software adoptada (un API gateway y siete microservicios independientes: seguridad, dispositivos, mediciones, hogares, notificaciones, alertas y recomendaciones), la infraestructura de alojamiento y los costos del proyecto. La solución integra un dispositivo ESP32 con sensor PZEM-004T V3, servicios en Java 21 con Spring Boot 4.1.1, una base de datos MySQL 8.4, una cola de mensajes RabbitMQ 4.1, un intermediario de mensajes Mosquitto con TLS, una aplicación web en React 19 y una aplicación móvil Android con Expo SDK 57. El sistema está desplegado en `https://energy-monitor.tech`, en un VPS Hostinger KVM 2 con Docker Compose, y se actualiza mediante integración y despliegue continuos con GitHub Actions. El costo total estimado es de COP 57.021.997, de los cuales COP 8.073.997 corresponden a hardware, COP 48.000 a infraestructura (un mes de servidor; el dominio se obtuvo sin costo) y COP 48.900.000 a recurso humano. El documento incluye además el cronograma, el estado por componente, los riesgos técnicos pendientes y las diferencias con el documento general del proyecto.

*Palabras clave:* propuesta técnica, arquitectura, microservicios, API gateway, costos, infraestructura, integración continua, Internet de las cosas

# Introducción

EnergyMonitor es un proyecto formativo del programa Tecnólogo en Análisis y Desarrollo de Software (ADSO) del SENA, desarrollado por dos aprendices con fecha de entrega el 28 de octubre de 2026. Su finalidad es que los hogares vean cuánta energía consume cada electrodoméstico, detecten los momentos de mayor demanda, reciban alertas por consumo inusual y obtengan recomendaciones de ahorro basadas en sus propias mediciones.

Esta propuesta responde a la pregunta de si el proyecto es técnica y económicamente viable con los recursos del equipo. Se organiza en tres bloques: la justificación de la pila tecnológica, la arquitectura de software y la infraestructura con sus costos. Después presenta el cronograma y el estado por componente, los riesgos técnicos y las diferencias con el documento general del proyecto. Los requisitos funcionales y no funcionales se detallan en la especificación de requisitos (carpeta `02-especificacion-requisitos`); los objetivos, el alcance y el equipo, en el documento general del proyecto (carpeta `01-documento-general-proyecto`); y el procedimiento de despliegue, en el manual de implementación y despliegue (carpeta `09-implementacion`).

# Justificación de la Pila Tecnológica

## Visión general de la solución

El flujo de datos recorre cinco piezas. El dispositivo ESP32 lee el sensor PZEM-004T V3 por Modbus RTU a 9600 baudios (GPIO16 como recepción y GPIO17 como transmisión, firmware 1.3.8) y publica por MQTT 3.1.1 con TLS en el puerto 8883 hacia el intermediario Mosquitto. Los servicios en Java reciben las lecturas: el servicio de mediciones las almacena en MySQL y anuncia cada una, los servicios de alertas y de recomendaciones las evalúan y el de notificaciones las entrega. La aplicación web y la aplicación móvil Android consultan los servicios mediante una API REST, siempre a través del API gateway. El dispositivo transmite telemetría cada 60 segundos y su estado cada cinco minutos. La vinculación se realiza por Bluetooth Low Energy desde la web, con un portal de configuración como alternativa. En producción, la web y la API se publican en `https://energy-monitor.tech` y los dispositivos se conectan a `mqtt.energy-monitor.tech:8883`.

## Decisiones por capa

La Tabla 1 resume la tecnología elegida en cada capa y la razón que la respalda. La última columna compara cada elección con alternativas comunes del mercado. Es una comparación de referencia, no el registro de evaluaciones formales hechas durante el proyecto.

**Tabla 1**

*Tecnología Elegida y Justificación por Capa*

| Capa | Tecnología elegida | Justificación | Alternativas de mercado |
|---|---|---|---|
| Servicios | Java 21, Spring Boot 4.1.1, Spring Security, Spring Data JPA; Spring Cloud Gateway Server Web MVC (Spring Boot 4.0.8, Spring Cloud 2025.1.3) en el API gateway | Herramientas de software libre con eficiencia, escalabilidad y facilidad de integración. Spring Modulith separó el servicio central original en módulos con dependencias controladas y permitió extraer cada contexto a un servicio propio; Spring Cloud Gateway da una única entrada a la API con la misma pila. | Node.js con NestJS o Express: ecosistema amplio y desarrollo rápido, pero sin la modularidad verificable de Spring Modulith. Django (Python) y ASP.NET Core (C#): válidos, con menos experiencia en el equipo. Gateways como Kong o NGINX: otra tecnología que mantener. |

| Capa | Tecnología elegida | Justificación | Alternativas de mercado |
|---|---|---|---|
| Mensajería entre servicios | RabbitMQ 4.1 con *transactional outbox* | Software libre. La fila del mensaje se escribe en la misma transacción que el cambio y un relay la publica con confirmación del broker, de modo que ningún evento se pierde al separar servicios. | Kafka: más capacidad y complejidad de la necesaria para un solo servidor. Llamadas HTTP síncronas para todo: acoplan la disponibilidad de los servicios. |
| Base de datos | MySQL 8.4 con migraciones Liquibase | Software libre. Liquibase versiona los cambios de la base de datos con reversión (rollback), lo que da trazabilidad. | PostgreSQL: igual de libre y más rico en funciones avanzadas, innecesarias para este volumen. MongoDB: flexible, pero los datos del proyecto son relacionales (hogares, miembros, dispositivos). Firebase: costo variable por uso y dependencia del proveedor. |
| Mensajería con el dispositivo | MQTT 3.1.1 con TLS sobre Eclipse Mosquitto | Protocolo estándar de Internet de las cosas (OASIS, 2014), apto para dispositivos pequeños. Mosquitto es libre y los certificados se obtienen con Let's Encrypt. El cifrado protege las lecturas en tránsito. | HTTP periódico desde el dispositivo: más simple, pero consume más recursos del microcontrolador y no tiene mecanismos de última voluntad. CoAP: menos soporte en brokers y herramientas. Brokers como EMQX o HiveMQ: más capacidad de la necesaria para un solo servidor. |
| Dispositivo | ESP32 con sensor PZEM-004T V3 | Microcontrolador con Wi-Fi y Bluetooth de bajo costo (COP 24.000) y sensor de energía de COP 70.000. Registra tensión, corriente, potencia activa y energía acumulada. | Arduino con módulo Wi-Fi o Raspberry Pi: el primero exige más componentes y la segunda tiene mayor costo y consumo. Sensores como el SCT-013 (pinza de corriente) requieren calibración y circuito propio, mientras que el PZEM-004T entrega los valores ya calculados. |
| Aplicación web | React 19, Vite 8, Zustand, Axios, React Router, React Hook Form, Zod, i18next, Recharts | Software libre. Permite gráficos interactivos, formularios validados e interfaz en varios idiomas. Web Bluetooth habilita la vinculación directa del dispositivo desde el navegador. | Angular o Vue: igual de viables, con menos conocimiento previo en el equipo. Next.js: añade renderizado en servidor, que la aplicación no necesita. Create React App: en desuso frente a Vite. |
| Aplicación móvil | React Native 0.86 con Expo SDK 57, expo-router, Redux Toolkit | Software libre. Reutiliza el conocimiento de React del equipo y atiende solo Android, que es el alcance definido. | Flutter (Dart): buen rendimiento, pero obliga a otro lenguaje y a no reutilizar lo aprendido en la web. Kotlin nativo: máximo control en Android, con mayor costo de desarrollo. Una aplicación web progresiva: menor acceso a funciones del teléfono. |
| Infraestructura | Docker Compose sobre un VPS de Hostinger KVM 2, Nginx con certificados Let's Encrypt e integración y despliegue continuos con GitHub Actions | Despliegue reproducible en un único servidor de bajo costo. El mismo servidor aloja el API gateway, los servicios, la base de datos, la cola de mensajes, el intermediario y la aplicación web. GitHub Actions ya está integrado con los repositorios del equipo y permite aprobar cada despliegue. | AWS, Google Cloud o Azure: más servicios y escalado automático, con costo variable y mayor complejidad. Plataformas PaaS: muchas solo publican HTTP y no admiten el puerto 8883 de MQTT. Otros VPS (DigitalOcean, Contabo): comparables. Jenkins o GitLab CI: exigen otro servidor o migrar los repositorios. |

## Criterios generales de selección

Las decisiones de la Tabla 1 comparten tres criterios que orientaron la selección:

- Costo de licencias igual a cero: todo el software de la pila es libre, y solo se paga el servidor. El dominio se obtuvo sin costo.
- Un solo servidor: la carga prevista de un proyecto formativo cabe en un VPS. Los microservicios corren en el mismo servidor con Docker Compose y comparten una sola base MySQL; el sistema puede crecer aumentando los recursos del servidor.
- Conocimiento del equipo: la pila usa herramientas que los dos integrantes manejan, lo que reduce el riesgo de entrega dentro del plazo.

### Limitaciones asumidas

Dos decisiones tienen límites conocidos. Web Bluetooth solo funciona en navegadores compatibles, como Chrome, y exige un contexto seguro (en producción lo da HTTPS); en los demás navegadores se usa el portal de configuración Wi-Fi. La aplicación móvil se limita a Android, por lo que no hay versión para iPhone.

## Arquitectura de Software

### Patrón arquitectónico elegido

EnergyMonitor nació como un monolito modular construido con Spring Modulith: una sola aplicación desplegable, dividida en siete módulos con límites claros. Entre el 24 y el 28 de octubre de 2026 esa aplicación se repartió en servicios independientes: seguridad, dispositivos y mediciones el 24 de octubre, y hogares, notificaciones, alertas y recomendaciones el 28 de octubre, con la migración de los cuatro últimos a cargo de Miguel Ángel García Artunduaga. Cada servicio tiene su repositorio, su contenedor y su pipeline, y delante de todos hay un API gateway, única entrada de la API. El servicio central (`energy-monitor-backend`) se retiró. La extracción no cambió las rutas públicas de la API para los clientes. No se usa Kafka. La Tabla 2 describe cada componente.

**Tabla 2**

*Servicios y Módulos del Sistema*

| Componente | Responsabilidad |
|---|---|
| API gateway (`en-monitor-api-gateway`, puerto 8080) | Única entrada de la API. Responde CORS, valida el token (RS256, emisor y expiración) salvo en los endpoints públicos, lee el token de la cookie HttpOnly de la web, limita los POST públicos de cuenta (20 por IP y ruta por minuto), rechaza `/internal/**` y responde 503 si un servicio no responde o 504 si tarda más de 30 s |
| Servicio de seguridad (`en-monitor-security-service`, puerto 8082) | Registro con verificación previa del correo y aceptación de términos, autenticación, Google Sign-In, JWT y refresh token rotativo, sesiones en cookies HttpOnly para la web, recuperación de contraseña, perfil, borrado de cuenta (con código por correo para las cuentas de Google), RBAC y administración global de cuentas y roles (rol `ADMIN`), contactos de soporte |
| Servicio de dispositivos (`en-monitor-device-service`, puerto 8084) | Vinculación, edición, reemisión de credenciales y desvinculación de dispositivos, catálogo de electrodomésticos, autenticación y autorización MQTT, comandos a los módulos y conectividad (sin conexión tras 150 s sin datos) |
| Servicio de mediciones (`en-monitor-measurement-service`, puerto 8085) | Único que guarda la telemetría; historial, última lectura, estadísticas, consumo de un hogar y niveles de consumo; anuncia cada lectura |
| Servicio de hogares (`en-monitor-home-service`, puerto 8086) | Hogares, tipos de hogar, miembros, roles OWNER y MEMBER, favoritos, umbrales, eliminación del hogar y transferencia de propiedad; responde las verificaciones de borrado de cuenta y de pertenencia a un hogar que hacen los demás servicios |
| Servicio de notificaciones (`en-monitor-notification-service`, puerto 8087) | Entregas, preferencias, correo electrónico (el único que envía correo), Web Push, push móvil, avisos in-app y actividad de hogares y dispositivos |
| Servicio de alertas (`en-monitor-alert-service`, puerto 8088) | Alertas de consumo, de conectividad y de consumo inusual (una lectura de al menos 200 W y 2,5 veces la potencia habitual del dispositivo en los 7 días anteriores) a partir de las lecturas y de los eventos de los dispositivos |
| Servicio de recomendaciones (`en-monitor-recommendation-service`, puerto 8089) | Reglas de ahorro, evaluadas cada hora con la energía por hora del servicio de mediciones |

### Reglas de organización

- Cada servicio es dueño de su dominio y de sus tablas, y consulta a los demás solo por su API interna o por eventos; ninguno lee las tablas de otro. `ModularityTests` (Spring Modulith) verificó esos límites mientras los

módulos vivían en el servicio central.

- Cada servicio y cada módulo aplican arquitectura hexagonal (`domain`, `application`, `adapter`, `infrastructure`, `api`). El gateway no tiene dominio.
- Todos los servicios usan una sola base MySQL 8.4 (`energy_monitor`); cada uno es dueño de sus tablas y aplica solo sus changesets de Liquibase, que incluyen rollback. No hay claves foráneas entre servicios.
- Los identificadores son VARCHAR(10) y el borrado es lógico mediante `deleted_at`.

## Comunicación entre módulos y servicios

Los servicios se consultan por REST interno bajo `/internal/**`, que solo se acepta desde la red de Docker y con el encabezado `X-Internal-Token`, y que el gateway nunca reenvía. Los cambios se anuncian con eventos en RabbitMQ 4.1 mediante *transactional outbox*: la fila del mensaje se escribe en la misma transacción que el cambio, un relay la publica cada segundo con confirmación del broker, la entrega es al menos una vez y las colas son durables, con *dead-letter* tras tres intentos. Todos los eventos entre servicios viajan por RabbitMQ. La Tabla 3 los enumera.

**Tabla 3**

*Eventos entre Módulos y Servicios*

| Evento | Origen | Destino | Canal |
|---|---|---|---|
| `measurement.recorded` | servicio de mediciones | servicio de alertas | Rabbit MQ |
| `device.connectivity-lost` / `device.connectivity-restored` | servicio de dispositivos | servicio de alertas | Rabbit MQ |
| `device.linked` / `device.unlinked` | servicio de dispositivos | servicio de mediciones (caché), servicio de alertas y servicio de notificaciones (actividad) | Rabbit MQ |
| `alert.raised` | servicio de alertas | servicio de notificaciones | Rabbit MQ |
| `recommendation.created` | servicio de recomendaciones | servicio de notificaciones | Rabbit MQ |
| `account.registered` / `account.administered` | servicio de seguridad | servicio de notificaciones | Rabbit MQ |
| `account.deleted` | servicio de seguridad | servicio de hogares y servicio de notificaciones (limpieza) | Rabbit MQ |
| `mail.email-verification` / `mail.password-reset` / `mail.account-deletion` | servicio de seguridad | servicio de notificaciones | Rabbit MQ |
| `home.deleted` | servicio de hogares | servicio de dispositivos y servicio de notificaciones | Rabbit MQ |
| `home.created`, `home.member-joined`, `home.member-left`, `home.member-removed`, `home.thresholds-updated`, `home.ownership-transferred` | servicio de hogares | servicio de notificaciones (actividad) | Rabbit MQ |

El servicio de recomendaciones no escucha eventos de medición: evalúa las recomendaciones cada hora mediante un programador (`RecommendationScheduler`) y pide al servicio de mediciones la energía por hora. La pérdida de conectividad de un dispositivo se identifica mediante los registros de estado y los mecanismos de última voluntad y silencio del protocolo MQTT; el servicio de dispositivos la anuncia con `device.connectivity-lost` y su recuperación con `device.connectivity-restored`. Al recibir `home.deleted`, el servicio de dispositivos desvincula los módulos del hogar y les envía el comando `unlink`. El registro de una cuenta y los cambios de estado o de rol que hace un administrador generan avisos in-app para los demás administradores, y la actividad de los hogares y los dispositivos llega a los usuarios por una sola cola de notificaciones. Solo el servicio de

notificaciones envía correo: el servicio de seguridad le pide por RabbitMQ los de verificación, recuperación y código de borrado de cuenta.

## Interfaces

La API es REST bajo `/api/v1` y cubre autenticación, cuentas, hogares, miembros, umbrales, dispositivos, mediciones, estadísticas, alertas, recomendaciones, preferencias de notificación y administración (`/api/v1/admin`: usuarios, roles y avisos, con los permisos `USER.READ`, `USER.MANAGE` y `ROLE.MANAGE`). El API gateway reparte las rutas: `/api/v1/auth/**`, `/api/v1/admin/users/**`, `/api/v1/admin/roles/**` y `/api/v1/support/**` al servicio de seguridad; `/api/v1/homes/{homeId}/devices/**` y `/api/v1/appliance-types/**` al de dispositivos; `/api/v1/measurements/**`, `/api/v1/consumption-levels/**` y `/api/v1/homes/{homeId}/consumption/**` al de mediciones; el resto de `/api/v1/homes/**` y `/api/v1/home-types/**` al de hogares; `/api/v1/notifications/**` y `/api/v1/admin/notifications/**` al de notificaciones; `/api/v1/alerts/**` al de alertas; y `/api/v1/recommendations/**` al de recomendaciones. Cualquier otra ruta responde 404. Cada servicio incluye Actuator y Swagger (el gateway, solo Actuator); en producción Swagger no pasa por el proxy. Existen además endpoints internos de autenticación y autorización MQTT para Mosquitto en el servicio de dispositivos, con caché de cinco minutos, que no se exponen a Internet.

## Seguridad

- Tokens JWT firmados con RS256. El access token dura 15 minutos.
- Refresh tokens opacos con estado, rotación y detección de reutilización, almacenados en la base de datos.
- Permisos globales consultados dinámicamente, sin roles dentro del JWT; los demás servicios los piden al servicio de seguridad por su API interna, con una caché breve.
- La web guarda la sesión en cookies HttpOnly que pone el servicio de seguridad; el gateway lee el token de esa cookie.
- La identidad se toma de la claim `sub` mediante CurrentUserResolver. El encabezado X-User-Id se ignora.
- CORS cerrado por defecto, con orígenes definidos por variables de entorno; lo responde el API gateway. En producción la web y la API comparten origen.
- Google Sign-In requiere credenciales OAuth configuradas. La cuenta creada con Google no tiene contraseña: no puede cambiarla y confirma su borrado con un código que llega a su correo.
- Ninguna cuenta entra sin aceptar los términos y condiciones (428 si falta); el registro con contraseña crea una solicitud pendiente de 24 horas y la cuenta solo existe al verificar el correo, cuyo dominio se comprueba (proveedores con error de escritura y dominios inexistentes se rechazan).
- Los dispositivos se autentican ante el intermediario con `device_code` y `api_key`; el servicio de dispositivos solo guarda el hash SHA-256 de la clave.
- Las llamadas internas entre servicios exigen la red de Docker y el encabezado `X-Internal-Token`, comparado en tiempo constante.
- En producción: HTTPS y MQTT con certificados Let's Encrypt, rutas `/internal/**` sin exposición pública y `X-Forwarded-For` reemplazado por la IP real en el proxy.
- La protección de los datos personales de los usuarios toma como referencia la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

## Reglas de dominio relevantes

Los roles globales los administra el servicio de seguridad, mientras que los roles OWNER y MEMBER son por hogar, de modo que un usuario puede pertenecer a varios hogares con roles distintos. El rol global `ADMIN` no da acceso a ningún hogar y un administrador no puede cambiar su propio estado ni sus roles. Los umbrales pertenecen al hogar, con valores predeterminados de 6 kWh diarios y 180 kWh mensuales. Los niveles de potencia son LOW (0 a 500 W), MEDIUM (500 a 1500 W), HIGH (1500 a 3000 W) y CRITICAL (desde 3000 W).

El borrado de cuenta es transaccional: valida la contraseña, cierra las sesiones, elimina lógicamente los datos personales, libera el correo y gestiona las membresías.

### Clientes

La aplicación web separa sus funcionalidades por características, gestiona los textos con i18n y concentra las llamadas HTTP en `src/services/`. Se prueba con Vitest y Testing Library. Una cuenta administradora ve solo la experiencia de administración (usuarios, notificaciones y ajustes). Ajustes ofrece la ayuda en modales: documentación según el rol, soporte técnico (solo para usuarios, que crean el ticket como un correo a un administrador) y novedades, que el build arma con los commits `feat`, `fix` y `perf` del historial de git, así que cada despliegue de la web las actualiza. La foto de perfil se amplía en un visor dentro de la página (la CSP no permite pedir `data:` con `fetch`) y los miembros de un hogar muestran sus fotos: el perfil interno de seguridad incluye `profileImage` y el servicio de hogares lo devuelve. La aplicación móvil usa la estructura de expo-router, archivos de estilos por componente y está disponible en español, inglés, francés y portugués. En el emulador de Android se usa 10.0.2.2 en lugar de localhost, y CORS no aplica en React Native.

### Convenciones de desarrollo

El código se escribe en inglés y los entregables académicos en español. Los commits siguen Conventional Commits con scope por módulo y mensaje en inglés imperativo. Las ramas son `feat/*`, `fix/*` o `chore/*`, con integración mediante pull request a `dev` y luego a `main` en todos los repositorios. Cada pull request debe pasar sus pruebas automáticas en GitHub Actions antes de fusionarse.

## Infraestructura y Costos

### Infraestructura de alojamiento

La Tabla 4 describe la infraestructura en producción.

**Tabla 4**

*Infraestructura de Alojamiento*

| Elemento | Detalle |
|---|---|
| Proveedor | Hostinger |
| Plan | VPS KVM 2: 2 vCPU, 8 GB de RAM y 100 GB NVMe, con Ubuntu 26.04 LTS |
| Contratación | Un mes, por COP 48.000. Con contrato anual el plan cuesta COP 27.900 mensuales |
| Despliegue | Docker Compose desde el repositorio `energy-monitor-infra`, con el API gateway, los siete servicios (seguridad, dispositivos, mediciones, hogares, notificaciones, alertas y recomendaciones), MySQL 8.4, RabbitMQ 4.1, Mosquitto y la aplicación web |
| Acceso | Nginx en el servidor como único punto de entrada HTTPS; solo se publican los puertos 80, 443 y 8883 |
| Certificados | Let's Encrypt, sin costo y con renovación automática |
| Dominio | `energy-monitor.tech` (web y API) y `mqtt.energy-monitor.tech` (intermediario), obtenido sin costo |
| Respaldos | Copia de MySQL diaria y antes de cada despliegue, conservada 14 días en el servidor |

### Integración y despliegue continuos

Los repositorios del API gateway, de los siete servicios, de la aplicación web y de la infraestructura tienen pipelines de GitHub Actions (el servicio de notificaciones incorporó el suyo el 28 de octubre de 2026); la aplicación móvil no tiene pipeline. En cada pull request se ejecutan las pruebas (al 28 de octubre de 2026: 774 del servicio de seguridad, 260 del de hogares, 235 del de notificaciones, 151 del de mediciones, 129 del de

dispositivos, 128 del de alertas, 71 del de recomendaciones y 69 del gateway, contra MySQL 8.4 los que lo necesitan; las 253 de la web con su build; y la validación de la configuración de infraestructura), que son requisito para fusionar. Cada cambio que llega a `main` se despliega en el servidor tras una aprobación manual del entorno `production`. El despliegue toma un respaldo de la base de datos, reconstruye solo el componente modificado, comprueba la salud de todos los servicios y, si falla, vuelve a la versión anterior. El detalle está en el manual de implementación y despliegue (Cortés Parra y García Artunduaga, 2026).

## Costo de hardware

La Tabla 5 detalla el hardware. Las primeras diez líneas corresponden al prototipo del dispositivo de medición, y las dos últimas a los equipos del equipo de desarrollo.

**Tabla 5**

*Costo de Hardware (COP)*

| Concepto | Cantidad | Costo unitario | Subtotal |
|---|---|---|---|
| ESP32 con Wi-Fi y Bluetooth | 1 | 24.000 | 24.000 |
| Medidor PZEM-004T-100A V3 | 1 | 70.000 | 70.000 |
| Fuente AC-DC HLK-PM03 | 1 | 20.000 | 20.000 |
| Cables jumper macho-macho de 20 cm | 40 | 300 | 12.000 |
| Cables jumper macho-hembra de 20 cm | 40 | 300 | 12.000 |
| Cable encauchetado 2x14 AWG | 3 | 3.333 | 9.999 |
| Cable calibre 14 rojo y negro | 1 | 2.400 | 2.400 |
| Conector JST XH2.54 | 1 | 1.300 | 1.300 |
| Cables jumper hembra-hembra de 20 cm | 6 | 383 | 2.298 |
| Caja plástica (carcasa del prototipo) | 1 | 20.000 | 20.000 |
| Computador del equipo de desarrollo | 2 | 3.500.000 | 7.000.000 |
| Celular del equipo de desarrollo | 2 | 450.000 | 900.000 |
| **Total de hardware** | | | **8.073.997** |

De este total, COP 173.997 corresponden al prototipo del dispositivo de medición. Es el gasto directo en componentes, cercano a los COP 180.000 que indica el documento general del proyecto.

## Costo de software, licencias e infraestructura

Todo el software de la pila es libre y no genera costo de licencia. La Tabla 6 separa lo gratuito de lo que se paga.

**Tabla 6**

*Costo de Software, Licencias e Infraestructura (COP)*

| Concepto | Tipo | Costo |
|---|---|---|
| Java 21 (OpenJDK), Spring Boot, Spring Modulith, Spring Security, Spring Data JPA, Spring Cloud Gateway | Libre | 0 |
| RabbitMQ | Libre | 0 |
| MySQL 8.4 y Liquibase | Libre | 0 |
| Eclipse Mosquitto | Libre | 0 |
| Docker, Docker Compose y Nginx | Libre | 0 |
| React 19, Vite 8, Zustand | Libre | 0 |
| Expo SDK 57 y React Native 0.86 | Libre | 0 |
| Arduino IDE y Visual Studio Code | Libre | 0 |
| Let's Encrypt (certificados TLS) | Libre | 0 |

| Concepto | Tipo | Costo |
|---|---|---|
| GitHub y GitHub Actions | Gratuito | 0 |
| Dominio `energy-monitor.tech` | Gratuito | 0 |
| Hostinger VPS KVM 2, contrato de un mes | Pago | 48.000 |
| **Total de software e infraestructura** | | **48.000** |

El precio del plan depende del plazo de contratación. El proyecto lo contrató por un mes, suficiente para la demostración y la sustentación. Para una operación continua conviene el contrato anual: COP 27.900 mensuales, equivalentes a COP 334.800 por año, frente a COP 576.000 si se renueva mes a mes. El dominio gratuito debe revisarse antes de su vencimiento, porque la renovación puede tener costo.

## Servicios de terceros

La Tabla 7 recoge los servicios externos que el sistema usa. En el volumen actual del proyecto ninguno genera costo.

**Tabla 7**

*Servicios de Terceros*

| Servicio | Uso | Costo |
|---|---|---|
| Let's Encrypt | Certificados TLS | 0 |
| Google Sign-In | Inicio de sesión con Google en la web | 0 |
| Gmail (SMTP con contraseña de aplicación) | Envío de verificación, recuperación y notificaciones | 0 |
| Web Push (VAPID) | Notificaciones en el navegador | 0 |
| Expo Push | Notificaciones en la aplicación móvil | 0 |
| GitHub Actions | Pruebas y despliegue continuo | 0 |

## Costo de recurso humano

La Tabla 8 valora el trabajo del equipo durante siete meses. Es un costo estimado para dimensionar el proyecto, no un pago efectivo.

**Tabla 8**

*Costo Estimado de Recurso Humano (COP)*

| Rol | Meses | Costo mensual | Subtotal |
|---|---|---|---|
| Líder de proyecto | 7 | 2.000.000 | 14.000.000 |
| Desarrollador de software | 7 | 2.500.000 | 17.500.000 |
| Gestor de bases de datos | 4 | 1.200.000 | 4.800.000 |
| Analista | 7 | 1.800.000 | 12.600.000 |
| **Total de recurso humano** | | | **48.900.000** |

## Presupuesto consolidado

**Tabla 9**

*Presupuesto Consolidado del Proyecto (COP)*

| Categoría | Subtotal | Participación |
|---|---|---|
| Hardware | 8.073.997 | 14,16 % |
| Software e infraestructura | 48.000 | 0,08 % |
| Recurso humano (7 meses) | 48.900.000 | 85,76 % |

| Categoría | Subtotal | Participación |
|---|---|---|
| Otros | 0 | 0,00 % |
| **Total estimado** | **57.021.997** | **100 %** |

Casi el 86 % del costo es trabajo del equipo. Excluido el recurso humano, la inversión en equipos, prototipo y servidor es de COP 8.121.997, y la que realmente debe pagarse de forma recurrente es el servidor.

### Costos para los usuarios

El usuario no paga licencias de software. Necesita un celular Android o un navegador moderno. La Tabla 10 muestra rangos de referencia de los equipos.

**Tabla 10**

*Rangos de Precio de Equipos de Referencia para Usuarios (COP)*

| Equipo | Rango de precio |
|---|---|
| Celular Xiaomi Redmi 13 | 450.000 a 700.000 |
| Celular Samsung Galaxy A55 | 1.600.000 a 2.200.000 |
| Portátil HP 240 G9 (Core i5, 8 GB, 512 GB SSD) | 2.500.000 a 2.900.000 |
| Portátil Dell Inspiron 15 (Core i5, 8 GB) | 1.800.000 a 2.500.000 |

El costo del dispositivo de medición para un hogar es el de su prototipo, cercano a COP 174.000 en componentes. Un precio de venta o producción en serie queda fuera del alcance.

### Mantenimiento y licenciamiento

El plan de mantenimiento contempla tres modalidades. El correctivo atiende fallos de comunicación, carga, notificaciones e historial; un despliegue fallido se revierte de forma automática. El preventivo incluye actualización de dependencias (Spring Boot, React, Expo, firmware), los respaldos automáticos de MySQL (diarios y antes de cada despliegue), la renovación automática de los certificados y pruebas de seguridad. El evolutivo considera medidores inteligentes, actualización del firmware por aire (hoy se hace por cable USB), la ampliación de la interfaz de administración (gestión de soporte y de contenidos) y adaptación a nuevas versiones de Android y de los navegadores. Durante el primer mes posterior al despliegue se presta soporte correctivo.

El software y el hardware se licencian bajo una modalidad de uso libre con reconocimiento comunitario: se pueden usar, copiar y modificar con fines educativos, investigativos y sociales, manteniendo la referencia al equipo y al SENA. El uso comercial requiere un acuerdo formal entre los desarrolladores y el SENA, y puede tener una tarifa de licencia.

## Cronograma y Estado Actual

### Cronograma

El proyecto inició el 1 de mayo de 2026 y se entrega el 28 de octubre de 2026, fecha fijada para la evaluación; la puesta en producción se hizo entre el 16 y el 20 de octubre, antes de la entrega, y la arquitectura de servicios llegó a producción el 24 de octubre (seguridad, dispositivos y mediciones) y el 28 de octubre (hogares, notificaciones, alertas y recomendaciones). La Tabla 11 muestra las fases.

**Tabla 11**

*Fases y Fechas del Proyecto*

| Fase | Periodo | Resultado esperado |
|---|---|---|
| 1. Base y ciclo principal | 1 de mayo a 30 de junio | Un usuario se registra, crea un hogar, vincula un dispositivo y ve llegar mediciones |

| Fase | Periodo | Resultado esperado |
|---|---|---|
| 2. Visibilidad del consumo y alertas | 1 de julio a 31 de agosto | El consumo es visible y el sistema avisa cuando algo es anormal |
| 3. Recomendaciones e internacionalización | 1 a 30 de septiembre | Recomendaciones según el consumo propio e interfaz en más de un idioma |
| 4. Consolidación | 1 a 16 de octubre | Sistema listo para demostración y sustentación |
| 5. Puesta en producción y entrega | 16 a 28 de octubre | Sistema publicado en `https://energy-monitor.tech` con integración y despliegue continuos (16 a 20 de octubre), API gateway y microservicios de seguridad, dispositivos y mediciones en producción (24 de octubre), migración completa de los demás módulos y retiro del servicio central (28 de octubre) y entrega con sustentación el 28 de octubre |

## Estado por componente

La Tabla 12 presenta el avance al 28 de octubre de 2026.

**Tabla 12**

*Estado por Componente al 28 de Octubre de 2026*

| Componente | Estado |
|---|---|
| Dispositivo de medición (firmware 1.3.8) | Operativo. La vinculación de un medidor en producción está pendiente de evidencia |
| API gateway | Desplegado en producción; 69 pruebas automáticas en cada pull request |
| Servicio de seguridad | Desplegado en producción; 774 pruebas automáticas |
| Servicio de dispositivos | Desplegado en producción; 129 pruebas automáticas |
| Servicio de mediciones | Desplegado en producción; 151 pruebas automáticas |
| Servicio de hogares | Desplegado en producción el 28 de octubre de 2026; 260 pruebas automáticas |
| Servicio de notificaciones | Desplegado en producción el 28 de octubre de 2026; 235 pruebas automáticas |
| Servicio de alertas | Desplegado en producción el 28 de octubre de 2026; 128 pruebas automáticas |
| Servicio de recomendaciones | Desplegado en producción el 28 de octubre de 2026; 71 pruebas automáticas |
| Aplicación web | Desplegada en producción en `https://energy-monitor.tech`; 253 pruebas automáticas |
| Aplicación móvil Android | En desarrollo, integrada con la API; 78 pruebas unitarias en 8 suites; fuera del despliegue en producción |
| Infraestructura | VPS Hostinger KVM 2 con Docker Compose, Nginx, Let's Encrypt, RabbitMQ y GitHub Actions; pruebas de punta a punta de los servicios sobre la pila local (`device-flow` 24, `measurement-flow` 28, `alert-flow` 36, `activity-flow` 60 y `account-deletion-flow` 12, todas en verde) |

Queda pendiente la gestión del historial del dispositivo según el periodo de vinculación. La migración a microservicios terminó el 28 de octubre de 2026. En producción faltan las evidencias funcionales que señala el manual de implementación y despliegue: medidor vinculado, correo, administración, respaldo programado y renovación de certificados.

# Riesgos Técnicos Pendientes

**Tabla 13**

*Riesgos Técnicos Pendientes*

| Código | Riesgo | Efecto |
|---|---|---|
| RT-01 | Mitigado: la `api_key` del dispositivo ya no se guarda en claro; solo se conserva su hash SHA-256 (`api_key_hash`, changesets `device-007` y `device-008`) | Las claves de dispositivos ya vinculados se renuevan al volver a vincularlos |
| RT-02 | La telemetría usa MQTT con QoS 0 | Posible pérdida de lecturas. Se evaluará QoS 1 |
| RT-03 | Mitigado: la web ya no guarda los tokens en localStorage; la sesión viaja en cookies HttpOnly que pone el servicio de seguridad y que ningún script puede leer | Riesgo de robo por XSS reducido |
| RT-04 | No hay actualización del firmware por aire (OTA) | El dispositivo se actualiza solo con cable |
| RT-05 | Los límites de intentos de autenticación están en memoria | Se reinician al reiniciar el servicio |
| RT-06 | Parcialmente mitigado: las pruebas de persistencia requieren una base de datos activa, pero en GitHub Actions se ejecutan contra un MySQL 8.4 de servicio | En local sigue siendo necesario un MySQL propio |
| RT-07 | El access token sigue válido hasta 15 minutos tras borrar la cuenta | Ventana breve de acceso residual |
| RT-08 | Algunos casos límite de alertas de conectividad no se resuelven solos | Alertas que requieren revisión manual |
| RT-09 | Web Bluetooth tiene limitaciones de compatibilidad y exige contexto seguro | Se debe usar el portal de configuración en otros navegadores; en producción el contexto seguro lo da HTTPS |
| RT-10 | Los respaldos de MySQL se guardan en el mismo servidor que la base de datos | Una falla del servidor pierde los datos y sus respaldos a la vez |
| RT-11 | No hay monitoreo de disponibilidad ni alertas en producción | Una caída solo se detecta durante un despliegue o por aviso de un usuario |

# Discrepancias Documentales

La Tabla 14 señala las diferencias entre lo que dicen otros documentos del proyecto y lo que está implementado o desglosado en esta propuesta. Se verificaron contra el código del servicio central y de la aplicación web. Esta propuesta adopta siempre el estado implementado.

**Tabla 14**

*Diferencias Detectadas y Acción Propuesta*

| Código | Fuente de la diferencia | Lo que dice | Estado verificado | Acción |
|---|---|---|---|---|
| DD-01 | README de la aplicación web | Las recomendaciones son simuladas porque el módulo `recommendation` del servicio central no existe | El servicio de recomendaciones (antes, módulo `recommendation` del servicio central) existe y evalúa cinco reglas de ahorro | Corregir el README de la aplicación web. Sigue abierta al 20 de octubre de 2026 |
| DD-02 | Documento general del proyecto | Decía: inversión de unos COP 180.000 sin costo total | Presupuesto total estimado de COP 57.021.997. Los COP 180.000 equivalen al gasto directo en componentes del prototipo (COP 173.997) | Resuelta: la versión 1.1 del documento general incluye el costo total |
| DD-03 | Documento general del proyecto | Decía: alojamiento y servicios de pago no contemplados | VPS Hostinger KVM 2 de pago (COP 48.000 por un mes) y dominio gratuito | Resuelta: la versión 1.1 del documento general refleja la infraestructura de pago |

## Referencias

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Cortés Parra, J. E., & García Artunduaga, M. A. (2026). *Manual de implementación y despliegue de EnergyMonitor*. SENA – CIES Regional Huila.

OASIS. (2014). *MQTT Version 3.1.1* (OASIS Standard). https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html